

目录

第一章 问题诊断与设计哲学	4
1.1 现有系统的核心问题	4
1.2 设计哲学：文档驱动而非数值驱动	4
1.3 从"聊天机器人"到"叙事引擎"	4
第二章 核心架构总览	5
2.1 系统架构图	5
2.2 数据流与交互模式	5
2.3 可配置性原则	6
第三章 世界状态系统	6
3.1 设计目标	6
3.2 时间线系统	6
3.3 场景系统	7
3.4 全局状态管理	7
第四章 角色配置系统	7
4.1 从硬编码到可配置	7
4.2 角色配置文件结构	8
4.3 角色加载与动态引用	8
4.4 角色配置验证	9
第五章 角色状态文档系统	9
5.1 设计理念	9
5.2 角色状态文档结构	9
5.3 状态更新机制	10
5.4 手动干预与调试	10
第六章 关系网络系统	10

6.1 从硬编码关系到动态关系	11
6.2 关系文档结构	11
6.3 关系更新与冲突管理	12
第七章 事件引擎	12
7.1 事件驱动叙事	12
7.2 三种触发机制	12
7.3 事件配置文件	13
第八章 情感引擎	14
8.1 设计目标	14
8.2 情感分析流程	14
8.3 情感约束规则	14
8.4 情感日志	15
第九章 教学协作引擎	15
9.1 从硬编码分工到动态分工	15
9.2 动态分工机制	15
9.3 共享教学日志	15
9.4 互相补位机制	16
第十章 考核配置系统	16
10.1 从硬编码考试到可配置考核	16
10.2 考核配置文件结构	16
10.3 竞赛配置	17
10.4 考核模拟引擎	18
第十一章 叙事 Prompt 系统	18
11.1 动态 Prompt 生成	18
11.2 Prompt 组装流程	19
11.3 叙事 Prompt 模板	19

第十二章 数据库设计	19
12.1 设计原则	19
12.2 核心数据表	19
12.3 角色配置表	20
12.4 考核配置表	20
第十三章 文件目录结构	20
13.1 项目目录总览	20
13.2 关键目录说明	22
第十四章 开发路线图	22
14.1 分阶段开发计划	22
14.2 Phase 1 详细任务	23
14.3 Phase 2 详细任务	23
14.4 Phase 3-4 详细任务	23
第十五章 新旧对比	23
15.1 架构对比	23
15.2 核心改进总结	24

第一章 问题诊断与设计哲学

1.1 现有系统的核心问题

当前 cos2edu 项目的本质问题可以概括为一句话：角色是"带人设的 ChatGPT"，每次对话从零开始，角色之间互不感知，没有故事、没有进度、没有情感。这种设计虽然在单次对话中能够呈现角色的语言风格，但完全无法支撑一个持续演进的叙事教育体验。

具体而言，现有系统存在以下五个层面的根本性缺陷：

问题层面	具体表现	影响
状态无记忆	每次对话独立，角色不记得上次聊了什么	无法建立师生间的信任递进关系
角色互不感知	三位老师彼此不知道对方教了什么	教学内容重复或遗漏，无法协作
无时间线	没有倒计时、没有进度感	学习缺乏紧迫感和目标感
情感静态	角色态度永远不变	互动缺乏真实感，用户难以投入
硬编码角色	角色数量、性格、分工全部写死在代码中	无法扩展、无法复用到其他场景

1.2 设计哲学：文档驱动而非数值驱动

本架构最关键的设计决策是用 Markdown 文档而非数值系统来驱动一切——角色状态、关系网络、教学日志全部是人类可读的文档，LLM 可以直接理解和生成，用户也可以随时查看和手动调整。这一选择基于以下三个核心理念：

第一，自然语言表达力远超数值。一个"好感度 73"的数值无法区分"欣赏但保持距离"和"亲密但有些不安"两种截然不同的情感状态。而一段自然语言描述——"最近觉得他学习很认真，但有时候太较真了，让人有点压力"——则包含了丰富的情感层次和微妙的态度倾向，LLM 可以据此生成更加真实、更有层次感的对话。

第二，人类可读意味着人类可干预。当系统行为偏离预期时，用户可以直接打开 Markdown 文件，修改角色的想法或关系描述，而不需要理解复杂的数值系统或调试数据库。这种透明性不仅降低了系统的维护成本，也赋予了用户对叙事走向的最终控制权。

第三，LLM 原生理解 Markdown。不需要额外的解析层或映射规则，LLM 可以直接读取角色状态文档并据此生成对话。这消除了"数值到行为"的映射难题——传统系统中，从好感度数值到具体对话行为的映射往往需要大量手工规则，而自然语言描述天然就是行为的直接指导。

1.3 从"聊天机器人"到"叙事引擎"

本架构的目标不是构建一个更好的聊天机器人，而是构建一个叙事引擎——一个能够持续生成连贯故事、推动角色发展、维持情感张力的系统。在这个系统中，每次对话都不是孤立的事件，而是故事的一个片段；每个角色都不是静态的人设，而是一个在时间中成长和变化的个体；每次教学都不是简单的知识传递，而是一场有情感投入的学习旅程。

这种从"聊天机器人"到"叙事引擎"的范式转变，要求我们在系统架构层面做出根本性的改变：引入世界状态来维持时间和空间的一致性，引入角色状态文档来追踪个体的内心变化，引入事件引擎来推动剧情发展，引入情感引擎来保证情感演进的合理性，引入教学协作引擎来协调多位老师的分工配合。

第二章 核心架构总览

2.1 系统架构图

整个系统由六大核心模块组成，它们通过世界状态这一中心枢纽相互连接，形成一个闭环的叙事教育引擎。下图展示了各模块之间的关系和数据流向：

层 次	模 块	核心职责	数据载体
基础设施层	世界状态系统	时间线、场景、全局状态管理	world_state.md
角色层	角色配置系统	可配置的角色定义与加载	characters/*.yaml
角色层	角色状态文档系统	持续更新的角色内心世界	states/*.md
关系层	关系网络系统	动态角色间关系追踪	relationships.md
驱动层	事件引擎	时间/条件/随机事件触发	events/*.yaml
驱动层	情感引擎	对话后情感分析与状态更新	emotion_log.md
协作层	教学协作引擎	动态分工、共享日志、互相补位	teaching_log.md
考核层	考核配置系统	可配置的考试/竞赛模拟	assessments/*.yaml
表达层	叙事 Prompt 系统	动态 Prompt 生成与上下文组装	运行时生成

表 1 系统架构层次总览

2.2 数据流与交互模式

系统的核心数据流遵循"输入 - 处理 - 输出 - 反馈"的闭环模式。当用户发起一次对话时，系统首先从世界状态中读取当前时间、场景和所有角色的状态文档，然后由叙事 Prompt 系统将这些信息组装成上下文，传递给 LLM 生成对话。对话完成后，情感引擎自动分析对话内容，更新相关角色的状态文档和关系网络，事件引擎检查是否有新事件被触发，教学协作引擎更新教学

日志。整个闭环确保了系统状态的持续演进和叙事的连贯性。

这种设计的关键优势在于：所有状态变更都通过 Markdown 文档的更新来实现，而不是通过数据库中的数值修改。这意味着每一次状态变更都是人类可读、可理解、可干预的。用户可以随时打开任何一个状态文档，了解角色的内心世界，甚至手动修改以引导叙事走向。

2.3 可配置性原则

本架构的另一个核心原则是全面的可配置性。角色数量、性格、分工不再是硬编码的常量，而是通过 YAML 配置文件定义的可变参数。考核系统同样如此——奖学金考试的金额、形式、时间，辩论赛的参赛队伍数量、赛制、奖励，都可以通过配置文件灵活调整。这意味着同一套系统可以支撑完全不同的故事背景和角色设定，从"清华三女生辅导经济学学生"到"任何你想象中的教育叙事场景"，只需要修改配置文件即可。

第三章 世界状态系统

3.1 设计目标

世界状态系统是整个架构的基础设施层，它的核心职责是维护一个所有对话共享的"世界"——有时间线、有场景、有角色状态和关系网络。没有世界状态，每次对话就像是在真空中发生的事件，无法与之前或之后的对话建立任何联系。世界状态系统确保了叙事的时空一致性，让每一次对话都成为连续故事的一个有机组成部分。

3.2 时间线系统

时间线是世界状态的核心维度。在本系统的故事背景中，时间线以"天"为单位，从第 1 天开始，到第 90 天结束（三个月的倒计时）。每一天都有其特定的叙事意义——第 1 天是搬入公寓的日子，第 7 天是第一次周测验，第 30 天是月度考核，第 85 天是奖学金考试，第 90 天是考试结果公布。时间线的推进不是自动的，而是由用户的学习活动触发的：完成一次学习对话，时间推进若干小时；完成一次练习，时间推进半天；参加一次考试，时间推进一整天。

时间线的数据结构如下：

```
# world_state.md 中的时间线部分
## 时间线
当前日期：第23天
当前时段：下午
倒计时：距奖学金考试还有62天
距辩论赛报名截止还有70天

## 重要时间节点
- 第1天：三位女生搬入公寓
```

- 第7天：第一次周测验
- 第30天：月度考核
- 第45天：期中模拟考试
- 第60天：辩论赛校内选拔
- 第85天：奖学金考试(笔试+面试)
- 第86-90天：考试结果公布与辩论赛准备

3.3 场景系统

场景系统管理对话发生的物理空间。不同的场景会影响角色的行为模式和对话风格——在公寓客厅里，对话更加随意亲密；在图书馆里，对话更加专注严肃；在咖啡厅里，对话更加轻松活泼。场景由事件引擎或用户选择触发切换，每次场景切换都会被记录在世界状态中，确保叙事的空间连贯性。

场景名称	氛围特征	适用活动	角色行为倾向
公寓客厅	温馨、随意、亲密	日常闲聊、课后复习	更放松、更私人化
公寓厨房	生活化、烟火气	做饭时的随性讨论	更自然、更生活化
图书馆	安静、专注、严肃	深度学习、考前冲刺	更严谨、更高效
咖啡厅	轻松、活泼、社交	小组讨论、头脑风暴	更开放、更创意
教室	正式、结构化	模拟考试、正式授课	更规范、更教学化
校园小径	自然、放松	散步时的非正式交流	更随性、更感性

表 2 场景定义与角色行为倾向

3.4 全局状态管理

全局状态是世界状态中不属于任何单一角色或关系的共享信息，包括当前日期、当前场景、已发生的事件列表、当前活跃的教学主题等。全局状态以 Markdown 文档的形式存储，每次对话开始时读取，对话结束时更新。这种设计确保了所有模块对"当前世界处于什么状态"有一致的理解，避免了不同模块之间的状态不一致问题。

全局状态的更新遵循"最小变更"原则——每次只修改确实发生变化的部分，而不是重写整个文档。这不仅提高了效率，也保留了状态变更的历史轨迹，便于回溯和调试。

第四章 角色配置系统

4.1 从硬编码到可配置

原有系统将三位角色（三月七、甘雨、刻晴）的身份、性格、教学分工全部硬编码在代码中。这种做法的问题显而易见：如果想要增加一位新角色，需要修改多个源文件；如果想要将系统复用到另一个故事背景，几乎需要重写整个项目。本架构将角色定义从代码中完全剥离，改为通过 YAML 配置文件来定义，实现了角色的可配置、可扩展、可复用。

4.2 角色配置文件结构

每个角色对应一个独立的 YAML 配置文件，存放在 `characters/` 目录下。配置文件包含角色的基本信息、性格设定、教学风格、初始状态和对话模板等。以下是一个完整的角色配置示例：

```
# characters/keqing.yaml
id: keqing
name: 刻晴
source: 原神
age: 19
department: 计算机系

personality:
  core_traits: [外冷内热, 严格高效, 追求完美]
  surface: 冷漠、严肃、不苟言笑
  inner: 关心他人但不善表达, 对认可的人会默默付出
  quirks: 喜欢用数据说话, 偶尔会冒出意外的冷幽默

teaching_style:
  approach: 方法派
  strengths: [知识体系梳理, 解题方法论, 效率优化]
  catchphrase: "方法对了, 事半功倍"
  weakness: 有时过于追求效率, 忽略学习者的感受

initial_state:
  mood: 警惕
  attitude_toward_user: 中立偏冷淡
  teaching_notes: []

voice:
  tone: 简洁、直接
  vocabulary: 偏正式, 偶尔夹杂技术术语
  speech_pattern: 短句为主, 逻辑清晰
```

4.3 角色加载与动态引用

系统启动时，角色加载器会扫描 `characters/` 目录下的所有 YAML 文件，将它们解析为角色对象并注册到世界状态中。其他模块通过角色 ID（如 `keqing`、`ganyu`、`march7th`）来动态引用角色，而不是通过硬编码的变量名。这意味着增加一个新角色只需要添加一个 YAML 文件，无需修改任何代码。

动态引用的实现方式是：所有需要引用角色的地方（场景系统、事件引擎、教学协作引擎等）都通过角色注册表（CharacterRegistry）来获取角色对象，而不是直接引用特定的角色变量。角色注册表提供了按 ID 查询、按标签筛选、遍历所有角色等接口，确保了角色引用的灵活性和可扩展性。

4.4 角色配置验证

每个角色配置文件在加载时都会经过严格的验证，确保包含所有必需字段（id、name、personality、teaching_style 等），且字段值的类型和格式符合规范。验证失败的角色会被跳过并记录错误日志，而不是导致系统崩溃。这种容错设计保证了系统在面对不完整或错误的配置时仍能正常运行。

第五章 角色状态文档系统

5.1 设计理念

角色状态文档是本架构最核心的创新之一。每个角色拥有一份持续更新的 Markdown 文档，记录其心情、想法、对其他角色的看法、教学笔记等。这不是数值化的好感度系统，而是自然语言描述的内心世界。LLM 在每次生成对话前会读取这份文档，在对话后会根据对话内容更新这份文档，从而实现角色状态的渐进式演进。

选择 Markdown 而非数据库来存储角色状态，是因为 Markdown 是 LLM 最自然的输入输出格式。LLM 可以直接理解一段关于角色心情的自然语言描述，并据此生成符合角色当前状态的对话；同样，LLM 也可以在分析完对话后，直接输出更新后的角色状态描述。这种“LLM 原生”的存储格式，消除了数值到行为的映射层，大大简化了系统的复杂度。

5.2 角色状态文档结构

每个角色的状态文档遵循统一的结构模板，但各部分的内容完全由 LLM 根据对话历史动态生成和更新。以下是状态文档的完整结构：

```
# states/keqing_state.md

## 基本信息
角色：刻晴
更新时间：第23天 下午

## 当前心情
今天他做练习的时候比昨天专注多了，看来昨天的批评起作用了。不过还是有点担心他是不是只是在硬撑，下次得注意观察一下。
```

对用户的看法

学习能力中等偏上，但容易分心。最近有进步的趋势，尤其是方法论方面。有时候太在意别人的评价，需要更多自信。

对其他角色的看法

- 三月七：太吵了，但确实能让气氛不那么沉闷。她的教学方式太随意，不过对初学者可能更友好。
- 甘雨：教学很细致，但有时候过于温柔，学生可能不会认真对待。她的理论功底确实扎实。

教学笔记

- 第20天：他对反向传播的理解有误区，需要从计算图的角度重新讲解
- 第22天：做题速度提升了，但准确率还不够
- 第23天：建议他做一套综合练习题

近期关注

他的学习状态在好转，但距离考试要求还有差距。需要加大练习量，同时注意不要让他压力过大。

5.3 状态更新机制

角色状态的更新由情感引擎在每次对话后自动触发。更新过程分为三个步骤：首先，情感引擎分析对话内容，提取情感变化信号（如角色是否感到开心、失望、担忧等）；然后，将这些信号与角色当前状态进行融合，生成更新后的状态描述；最后，将更新写入状态文档，同时保留变更日志以便回溯。

状态更新遵循“渐进合理”原则——角色的情感 and 态度变化必须是渐进的，不能在一次对话中从“冷淡”跳到“亲密”。情感引擎会检查提议的状态变更幅度，如果变更过大，会自动进行平滑处理，确保角色演进的连贯性和可信度。这种机制防止了角色性格的突变，维护了叙事的真实感。

5.4 手动干预与调试

由于角色状态以 Markdown 文档存储，用户可以随时直接编辑这些文档来调整角色的内心世界。这在以下场景中特别有用：当系统生成的状态更新不符合预期时，用户可以手动修正；当想要引导叙事走向特定方向时，用户可以修改角色的态度或想法；当想要重置某个角色的状态时，用户可以恢复到之前的状态版本。系统还提供了状态版本管理功能，每次更新都会保留历史版本，支持回滚到任意时间点。

第六章 关系网络系统

6.1 从硬编码关系到动态关系

原有系统中，角色之间的关系是隐含的、静态的——三位老师之间默认是朋友关系，与用户之间默认是师生关系。本架构将关系网络显式化、动态化，用独立的 Markdown 文档记录每一对角色之间的关系状态，并由情感引擎在每次对话后动态更新。

动态关系网络的核心价值在于：它允许角色间的关系随时间和互动自然演变。三位老师之间可能因为对用户关注度的不同而产生微妙的竞争心理，也可能因为教学理念的差异而发生争执，还可能因为共同面对困难而加深友谊。这些关系的动态变化为叙事提供了丰富的情感素材，也让互动更加真实有趣。

6.2 关系文档结构

关系网络以一个统一的 Markdown 文档存储，记录所有角色对之间的关系状态。每一对关系都包含关系类型、亲密度描述、近期互动摘要和特殊事件记录：

```
# relationships.md

## 刻晴 - 用户
类型：师生
亲密度：渐渐放下戒备，开始关心他的进步
近期互动：第22天因为学习态度问题批评了他，
    但第23天看到他进步后有些欣慰
特殊事件：无

## 甘雨 - 用户
类型：师生
亲密度：温柔耐心地引导，偶尔会因为他的
    进步慢而有些着急但不会表现出来
近期互动：第21天花了额外时间给他补基础
特殊事件：无

## 三月七 - 用户
类型：师生+朋友
亲密度：很聊得来，经常跑题聊别的
近期互动：第23天一起做练习时聊到了
    她最近追的番剧
特殊事件：无

## 刻晴 - 甘雨
类型：同事+朋友
亲密度：互相尊重但教学风格不同，
    偶尔会有分歧
近期互动：第20天讨论了教学进度安排
特殊事件：无

## 刻晴 - 三月七
类型：同事+朋友
```

亲密度：觉得她太吵但离不开她的活跃气氛
近期互动：第19天因为教学方式产生小争执
特殊事件：无

甘雨 - 三月七
类型：同事+朋友
亲密度：互补型朋友，甘雨包容三月七的活泼
近期互动：第22天一起准备了练习题
特殊事件：无

6.3 关系更新与冲突管理

关系的更新由情感引擎在每次对话后触发。当一次对话涉及多个角色时（例如用户和两位老师同时在场），情感引擎会同时更新所有相关角色对之间的关系状态。关系变更的幅度取决于对话中的情感强度和互动深度——一次深入的学术讨论可能微调彼此的尊重度，而一次情感冲突可能显著改变关系的亲密度。

系统特别关注角色间的负面情感（如失落、嫉妒、吃醋）的管理。这些情感是允许存在的，甚至是被鼓励的——它们增加了互动的真实感和情绪张力。但系统同时维护了一个"团队使命"约束：无论角色间的个人关系如何波动，在国际大赛夺冠这个团队目标面前，她们必须找到合作的方式。这种个人情感与团队使命之间的微妙平衡，是叙事张力的核心来源。

第七章 事件引擎

7.1 事件驱动叙事

事件引擎是推动剧情发展的核心动力。如果没有事件引擎，系统将陷入"用户问、角色答"的无限循环，缺乏叙事的起伏和转折。事件引擎通过三种触发机制——时间触发、条件触发和随机事件——来打破这种单调，为叙事注入意外性和戏剧性。

7.2 三种触发机制

时间触发事件

时间触发事件是在特定日期或时段自动发生的事件。它们构成了叙事的主干时间线，确保故事按照预定的节奏推进。例如：第 1 天三位女生搬入公寓，第 7 天第一次周测验，第 30 天月度考核，第 85 天奖学金考试。时间触发事件的定义存储在 YAML 配置文件中，可以灵活调整时间节点和事件内容。

时间节点	事件	叙事意义	影响范围
------	----	------	------

第1天	搬入公寓	故事起点，角色初次相遇	全部角色
第7天	第一次周测验	建立学习节奏	用户+全部老师
第14天	甘雨生日	增进感情的机会	甘雨+用户
第30天	月度考核	检验学习成果	用户+全部老师
第45天	期中模拟考试	暴露薄弱环节	用户+全部老师
第60天	辩论赛校内选拔	团队协作的起点	全部角色
第85天	奖学金考试	三个月努力的终极检验	用户+全部老师
第86-90天	结果公布与辩论赛准备	新篇章的开启	全部角色

表 3 主干时间线事件

条件触发事件

条件触发事件是当特定条件满足时自动发生的事件。它们为叙事提供了响应性——系统不是机械地按时间线推进，而是能够根据用户的实际学习进度和角色关系状态做出动态调整。例如：当用户连续三天没有学习时，触发"刻晴上门督促"事件；当某位老师的教学评价持续偏低时，触发"教学方式反思"事件；当两位老师对同一问题的教学方式产生分歧时，触发"教学理念碰撞"事件。

条件触发事件的定义采用"条件-动作"模式，条件可以是世界状态中的任何字段的组合（时间、场景、角色状态、关系状态等），动作可以是触发对话、切换场景、更新状态等。这种灵活的条件定义机制使得事件引擎能够响应各种复杂的叙事情境。

随机事件

随机事件是按一定概率在日常中发生的小插曲，它们为叙事增添了生活气息和不可预测性。随机事件不会改变叙事的主线走向，但会丰富角色的日常生活，创造更多自然互动的机会。例如：在咖啡厅偶遇某位老师、下雨天一起被困在公寓、收到家里寄来的包裹等。随机事件的概率和内容都可以通过配置文件调整，确保它们既不会太频繁以至于干扰学习，也不会太稀少以至于被遗忘。

7.3 事件配置文件

所有事件定义都存储在 events/ 目录下的 YAML 配置文件中，按触发机制分类组织。时间触发事件放在 events/timed_events.yaml 中，条件触发事件放在 events/conditional_events.yaml 中，随机事件放在 events/random_events.yaml 中。这种分类组织方式使得事件的维护和扩展变得清晰有序。

第八章 情感引擎

8.1 设计目标

情感引擎的核心目标是保证角色情感演进的渐进合理性。没有情感引擎，角色的情感状态要么永远不变（静态人设），要么在一次对话中发生不合理的剧变（LLM 幻觉）。情感引擎通过在每次对话后自动分析情感变化、约束变更幅度、更新状态文档，确保角色的情感演进既不僵化也不失控，而是在合理的范围内自然流动。

8.2 情感分析流程

每次对话结束后，情感引擎会执行以下四步分析流程：

第一步：对话摘要生成。LLM 首先对对话内容进行摘要，提取关键互动事件和情感信号。例如：“用户在讨论中表现出对反向传播的困惑，刻晴有些不耐烦但最终还是耐心地重新讲解了，甘雨在一旁补充了直观的解释。”

第二步：情感变化识别。基于对话摘要和角色当前状态，LLM 识别每个参与角色的情感变化方向和幅度。例如：“刻晴：从耐心到轻微不耐烦再到重新耐心，整体情感变化不大但增加了一丝担忧；甘雨：保持一贯的温和，对用户的困惑感到些许心疼。”

第三步：状态更新生成。LLM 根据情感变化识别的结果，生成更新后的角色状态描述和关系描述。更新遵循“渐进合理”原则——单次对话导致的情感变化幅度不超过当前状态的一个等级。例如，从“冷淡”可以变到“中立偏冷淡”，但不能直接跳到“友好”。

第四步：变更验证与写入。系统对 LLM 生成的状态更新进行验证，检查变更幅度是否在合理范围内、状态描述是否与角色性格一致等。验证通过后，更新写入角色状态文档和关系网络文档，同时记录变更日志。

8.3 情感约束规则

情感引擎实施以下约束规则，确保情感演进的合理性：

约束规则	描述	示例
渐进性约束	单次对话的情感变化不超过一个等级	冷淡->中立偏冷淡(允许), 冷淡->亲密(禁止)
性格一致性约束	情感变化方向必须与角色核心性格一致	刻晴不会突然变得黏人, 三月七不会突然冷漠
事件相关性约束	情感变化必须有对话中的具体事件支撑	不能无缘无故地改变态度
团队使命约束	个人情感不能完全破坏团队合作关系	即使吃醋也不会拒绝教学

回弹约束	极端情感状态会随时间自然回弹	暴怒后第二天会冷静一些
------	----------------	-------------

表 4 情感约束规则

8.4 情感日志

情感引擎的每次分析结果都会记录在 `emotion_log.md` 中，包括对话摘要、识别到的情感变化、生成的状态更新和验证结果。这份日志不仅是调试的重要工具，也是叙事回顾的宝贵资源——用户可以从中看到角色情感变化的完整轨迹，理解每一次态度转变背后的原因。

第九章 教学协作引擎

9.1 从硬编码分工到动态分工

原有系统将三位老师的分工硬编码为“甘雨理论派、刻晴方法派、三月七实践派”。虽然这种分工在初始设定下是合理的，但它缺乏灵活性——如果增加了一位新老师，她的教学风格如何融入现有的分工体系？如果某个主题更适合由不同的老师来讲解呢？本架构将教学分工改为动态的、基于角色配置的分工系统，使得教学协作能够根据实际需要灵活调整。

9.2 动态分工机制

动态分工的核心思想是：根据角色配置文件中的 `teaching_style` 字段，自动将教学内容分配给最合适的老师。分工不是固定的标签，而是基于教学风格匹配度的动态计算。例如，当一个学习主题是“理解反向传播的数学原理”时，系统会根据各角色的教学风格匹配度，推荐甘雨（理论派，擅长深入讲解原理）作为主讲，刻晴（方法派，擅长梳理推导步骤）作为补充，三月七（实践派，擅长代码实现）负责实践环节。

分工的动态性还体现在教学过程中的实时调整。如果教学日志显示某位老师的教学效果持续不佳（学生反复表示困惑），系统会自动建议调整分工，让更适合的老师接手。这种动态调整机制确保了教学质量的最优化，也反映了真实教学场景中老师之间的协作和互补。

9.3 共享教学日志

教学日志是三位老师共享的文档，记录了每次教学活动的内容、效果和后续建议。任何一位老师在开始教学前，都会先查阅教学日志，了解之前的进度和问题，从而避免重复教学或遗漏关键知识点。教学日志的存在使得三位老师的教学不再是各自为战，而是形成了一个连贯的、互补的教学体系。

```
# teaching_log.md
```

```
## 第23天 下午 - 刻晴授课
```

主题：卷积神经网络基础
内容：卷积操作原理、池化层、经典网络结构
效果：学生理解了基本概念，但对感受野
的计算还有困惑
建议：下次让甘雨从数学角度补充讲解，
三月七可以用代码演示

第22天 上午 - 甘雨授课
主题：反向传播算法深入
内容：计算图、链式法则、梯度消失问题
效果：学生对链式法则理解了，但梯度消失
的直觉还没建立
建议：刻晴可以从方法论角度总结解题技巧

9.4 互相补位机制

互相补位是教学协作的高级形态。当一位老师在教学中遇到困难（如学生反复表示不理解），系统会自动触发补位机制，建议另一位教学风格互补的老师介入。补位不是简单的替换，而是在保留原老师的教学主线的基础上，由补位老师提供不同角度的解释或补充。例如，刻晴的方法论讲解后学生仍然困惑，甘雨可以从理论角度补充直觉理解，三月七可以用代码实验来验证。这种“多角度攻克”的教学策略，远比单一老师的反复讲解更加有效。

第十章 考核配置系统

10.1 从硬编码考试到可配置考核

原有系统将考核方式硬编码为“奖学金考试+辩论赛”的固定组合。本架构将考核系统完全可配置化，通过 YAML 配置文件定义考核的类型、形式、时间、评分标准和奖励。这意味着同一套系统可以支撑各种不同的考核场景——从简单的课程考试到复杂的多轮竞赛，从个人挑战到团队对抗。

10.2 考核配置文件结构

考核配置文件存放在 `assessments/` 目录下，每个考核对应一个 YAML 文件。以下是奖学金考试的配置示例：

```
# assessments/scholarship_exam.yaml
id: scholarship_exam
name: AI精英奖学金考试
type: exam
schedule:
  date: day_85
  duration: 3_hours
```



```

format:
  - phase: written
    name: 笔试
    duration: 2_hours
    sections:
      - name: 选择题
        count: 20
        weight: 0.3
      - name: 简答题
        count: 5
        weight: 0.4
      - name: 编程题
        count: 2
        weight: 0.3
  - phase: interview
    name: 面试
    duration: 1_hour
    sections:
      - name: 知识深度考察
        weight: 0.4
      - name: 应用能力考察
        weight: 0.3
      - name: 综合素质考察
        weight: 0.3
reward:
  type: monetary
  amount: 1000000
  currency: CNY
  description: 100万人民币奖学金
impact:
  - triggers: debate_competition
    description: 获得奖学金后代表清华参加辩论赛

```

10.3 竞赛配置

竞赛是考核的高级形式，涉及多轮淘汰和团队对抗。以下是辩论赛的配置示例：

```

# assessments/debate_competition.yaml
id: debate_competition
name: 全国大学生"人工智能与未来社会"辩论赛
type: tournament
schedule:
  start_date: day_91
format:
  type: elimination
  rounds:
    - name: 四分之一决赛
  match_count: 4
  advancing: 4
  - name: 半决赛

```

```
match_count: 2
advancing: 2
- name: 决赛
match_count: 1
advancing: 1
teams: 8
team_size: 4
reward:
  national:
    type: opportunity
    description: 代表中国参加四国锦标赛
  international:
    type: combined
  monetary:
    amount: 1000000
    currency: USD
    per_person: true
  academic:
    offer: Stanford计算机系博士项目
sponsor: Anthropic
```

10.4 考核模拟引擎

考核模拟引擎负责在指定时间点模拟考核过程。对于考试类考核，引擎会根据用户的学习进度和教学日志，生成符合难度梯度的试题，并由 LLM 模拟笔试和面试过程。对于竞赛类考核，引擎会模拟对手团队的表现，生成辩论场景和对手论点，由 LLM 模拟整个辩论过程。考核结果会影响后续的叙事走向——通过考试意味着辩论赛篇章的开启，未通过则可能触发“再战一次”的支线剧情。

第十一章 叙事 Prompt 系统

11.1 动态 Prompt 生成

叙事 Prompt 系统是连接所有模块与 LLM 的桥梁。每次对话前，系统会动态组装一个包含完整上下文的 Prompt，而不是使用固定的模板。这个 Prompt 包含：当前世界状态（时间、场景）、参与角色的状态文档、相关角色对之间的关系描述、近期事件摘要、教学日志摘要，以及本次对话的叙事目标。

动态 Prompt 的核心优势在于：它确保 LLM 在每次对话中都拥有完整的上下文信息，从而生成与当前叙事状态一致的对话。LLM 不再需要“记住”之前发生了什么——因为所有相关信息都已经包含在 Prompt 中了。这种“无状态 LLM + 有状态 Prompt”的设计模式，是本架构实现叙事连贯性的关键技术。

11.2 Prompt 组装流程

Prompt 的组装遵循以下流程：首先，从世界状态中读取当前时间和场景；然后，根据场景和事件引擎的输出，确定本次对话的参与角色和叙事目标；接着，读取参与角色的状态文档和相关关系描述；最后，将所有信息按照预设的 Prompt 模板组装成完整的上下文 Prompt。整个过程是自动化的，用户无需手动干预。

11.3 叙事 Prompt 模板

叙事 Prompt 模板定义了上下文信息的组织方式和 LLM 的行为指导。模板中所有对角色的引用都是动态的——使用 {character_id} 占位符而非硬编码的角色名称，在运行时由系统根据角色配置替换为实际内容。以下是一个简化的 Prompt 模板示例：

```
你现在是{character.name}，{character.personality.core_traits}。  
当前场景：{scene.name}，{scene.atmosphere}  
当前时间：第{day}天 {time_period}  
  
你的当前心情：{character.state.mood}  
你对用户的看法：{character.state.attitude}  
你对其他在场角色的看法：{character.state.others}  
  
近期事件：{recent_events}  
教学进度：{teaching_log_summary}  
  
请以{character.name}的身份回应用户，  
保持角色性格的一致性，同时根据当前  
心情和关系状态调整互动方式。
```

第十二章 数据库设计

12.1 设计原则

虽然本架构的核心状态以 Markdown 文档存储，但仍然需要关系数据库来管理结构化数据——用户账户、对话历史、事件日志、考核记录等。数据库设计遵循“文档为主、数据库为辅”的原则：叙事相关的状态（角色状态、关系网络、教学日志）使用 Markdown 文档存储，便于 LLM 直接读写和用户手动干预；结构化的业务数据（用户信息、对话记录、考核成绩）使用数据库存储，便于查询和统计。

12.2 核心数据表

表名	用途	关键字段
----	----	------

users	用户账户	id, name, department, created_at
characters	角色注册表	id, name, source, config_path, is_active
conversations	对话记录	id, user_id, character_ids, scene, day, started_at
messages	消息记录	id, conversation_id, role, content, timestamp
events	事件日志	id, type, trigger, day, description, affected_characters
state_updates	状态变更日志	id, character_id, field, old_value, new_value, day, reason
teaching_sessions	教学记录	id, teacher_id, student_id, topic, day, effectiveness
assessments	考核定义	id, name, type, config_path, schedule_day
assessment_results	考核结果	id, assessment_id, user_id, score, passed, details
relationship_updates	关系变更日志	id, character_pair, change_description, day, trigger_event

表 5 核心数据表设计

12.3 角色配置表

characters 表是角色配置系统的数据库映射。它不存储角色的详细配置（这些在 YAML 文件中），而是存储角色的注册信息和元数据，包括角色的 ID、名称、来源、配置文件路径和是否激活。系统启动时，角色加载器会根据 characters 表中的记录，从对应的 YAML 配置文件中加载角色的详细配置。这种设计使得角色的注册管理和配置管理分离，便于批量操作和状态追踪。

12.4 考核配置表

assessments 表是考核配置系统的数据库映射。与角色配置类似，考核的详细配置存储在 YAML 文件中，数据库只存储注册信息和调度信息。assessments 表的关键字段包括考核 ID、名称、类型（exam/tournament 等）、配置文件路径和计划执行日期。考核模拟引擎在到达计划日期时，会根据配置文件自动触发考核模拟。

第十三章 文件目录结构

13.1 项目目录总览

以下是更新后的项目目录结构，新增了角色配置目录（characters/）、考核配置目录（assessments/）和角色状态目录（states/），原有的硬编码角色引用已全部替换为动态引用：

```
cos2edu/
+-- config/
| +-- app.yaml # 应用全局配置
| +-- world.yaml # 世界设定配置
| +-- prompts/ # Prompt 模板
| +-- narrative.yaml # 叙事 Prompt 模板
| +-- emotion_analysis.yaml # 情感分析 Prompt
| +-- state_update.yaml # 状态更新 Prompt
+-- characters/ # 角色配置目录(新增)
| +-- keqing.yaml # 刻晴配置
| +-- ganyu.yaml # 甘雨配置
| +-- march7th.yaml # 三月七配置
| +-- _template.yaml # 角色配置模板
+-- assessments/ # 考核配置目录(新增)
| +-- scholarship_exam.yaml # 奖学金考试
| +-- debate_competition.yaml # 辩论赛
| +-- _template.yaml # 考核配置模板
+-- states/ # 角色状态目录(新增)
| +-- keqing_state.md # 刻晴状态文档
| +-- ganyu_state.md # 甘雨状态文档
| +-- march7th_state.md # 三月七状态文档
| +-- world_state.md # 世界状态文档
| +-- relationships.md # 关系网络文档
| +-- teaching_log.md # 教学日志
| +-- emotion_log.md # 情感变更日志
+-- events/ # 事件配置目录
| +-- timed_events.yaml # 时间触发事件
| +-- conditional_events.yaml # 条件触发事件
| +-- random_events.yaml # 随机事件
+-- src/
| +-- core/
| | +-- world_state.py # 世界状态管理
| | +-- character_registry.py # 角色注册表
| | +-- scene_manager.py # 场景管理
| +-- engine/
| | +-- event_engine.py # 事件引擎
| | +-- emotion_engine.py # 情感引擎
| | +-- teaching_engine.py # 教学协作引擎
| | +-- assessment_engine.py # 考核模拟引擎
| +-- narrative/
| | +-- prompt_builder.py # Prompt 组装器
| | +-- context_manager.py # 上下文管理
| +-- storage/
| | +-- markdown_store.py # Markdown 文档存储
| | +-- db_store.py # 数据库存储
| | +-- version_control.py # 版本管理
+-- db/
```

```
| +-- schema.sql # 数据库表结构
| +-- migrations/ # 数据库迁移
+-- tests/
| +-- test_character_registry.py
| +-- test_emotion_engine.py
| +-- test_event_engine.py
| +-- test_teaching_engine.py
```

13.2 关键目录说明

characters/ 目录：存放所有角色的 YAML 配置文件。每个文件定义一个角色的完整属性，包括基本信息、性格设定、教学风格和初始状态。新增角色只需在此目录添加配置文件，系统会自动加载和注册。_template.yaml 提供了配置文件的模板和字段说明，方便用户快速创建新角色。

assessments/ 目录：存放所有考核的 YAML 配置文件。每个文件定义一个考核的完整属性，包括类型、时间、形式、评分标准和奖励。新增考核只需在此目录添加配置文件。_template.yaml 提供了考核配置的模板。

states/ 目录：存放所有动态状态文档。这些文档由系统在运行时自动更新，用户也可以手动编辑。每个角色对应一个状态文档，此外还有世界状态、关系网络、教学日志和情感日志等全局文档。版本管理系统会保留这些文档的历史版本，支持回滚和对比。

第十四章 开发路线图

14.1 分阶段开发计划

本项目的开发分为四个阶段，每个阶段聚焦于一个核心能力层的构建，确保系统可以渐进式地交付可用功能：

阶段	时间	核心目标	交付物
Phase 1 基础层	第1-3周	世界状态+角色配置+ Markdown 存储	可运行的世界状态系统，角色可配置加载，状态文档可读写
Phase 2 驱动层	第4-6周	事件引擎+情感引擎+ 教学协作引擎	事件可触发，情感可分析更新，教学可协作分工
Phase 3 表达层	第7-9周	叙事Prompt系统+ 考核模拟引擎	动态Prompt可组装，考核可模拟执行

Phase 4 完善层	第10-12周	集成测试+性能优化+ 文档完善	端到端测试通过，性能达标，用户文档完整
-------------	---------	-----------------	---------------------

表 6 分阶段开发计划

14.2 Phase 1 详细任务

Phase 1 是整个项目的基础，其核心任务是搭建世界状态系统、角色配置系统和 Markdown 存储系统。具体任务包括：实现 WorldState 类，支持时间线管理和场景切换；实现 CharacterRegistry 类，支持从 YAML 文件动态加载角色；实现 MarkdownStore 类，支持 Markdown 文档的读写和版本管理；实现状态文档的初始化逻辑，根据角色配置生成初始状态文档；编写单元测试，确保各模块的基本功能正确。

14.3 Phase 2 详细任务

Phase 2 在 Phase 1 的基础上构建三个驱动引擎。事件引擎需要实现三种触发机制（时间、条件、随机）和事件配置的加载与解析；情感引擎需要实现对话后的情感分析流程、状态更新生成和变更验证；教学协作引擎需要实现动态分工计算、共享教学日志和互相补位机制。此阶段还需要实现情感引擎与角色状态文档、关系网络的集成，确保情感分析的结果能够正确地更新到对应的文档中。

14.4 Phase 3-4 详细任务

Phase 3 聚焦于叙事表达层和考核模拟。叙事 Prompt 系统需要实现动态 Prompt 组装、上下文管理和 Prompt 模板渲染；考核模拟引擎需要实现考试模拟和竞赛模拟，包括试题生成、面试模拟、辩论场景生成和对手模拟。Phase 4 则是集成测试和完善阶段，重点确保所有模块的端到端集成正确、性能满足实时对话的要求、用户文档完整可用。

第十五章 新旧对比

15.1 架构对比

以下表格从多个维度对比了原有系统和本架构的差异，清晰地展示了本架构在各个层面的改进：

对比维度	原有系统	本架构
角色定义	硬编码在代码中	YAML配置文件，可扩展
角色状态	无（每次对话从零开始）	Markdown文档，持续更新

角色间感知	互不感知	通过关系网络和教学日志感知
时间线	无	90天倒计时，事件驱动
场景系统	无	6种场景，影响角色行为
情感系统	无（静态人设）	情感引擎，渐进合理演进
教学协作	各自独立教学	动态分工+共享日志+互相补位
考核系统	硬编码考试+辩论	YAML配置，支持多种考核类型
叙事连贯性	无（对话孤立）	世界状态+状态文档+事件引擎
可扩展性	低（修改需改代码）	高（添加配置文件即可）
可干预性	低（黑箱）	高（Markdown可读可编辑）
LLM上下文	固定Prompt模板	动态组装，包含完整世界状态

表 7 新旧架构对比

15.2 核心改进总结

本架构相对于原有系统的核心改进可以归纳为以下五个方面：

第一，从无状态到有状态。引入世界状态系统和角色状态文档，使得每次对话都建立在之前所有对话的累积之上，角色的情感和态度随时间自然演进，叙事具有真正的连贯性和发展性。

第二，从孤立到互联。引入关系网络和教学日志，使得角色之间不再是互不感知的独立个体，而是一个相互关联的社交网络。一位老师的教学会影响其他老师的教学策略，角色间的关系变化会影响彼此的互动方式。

第三，从静态到动态。引入事件引擎和情感引擎，使得叙事不再是用户单方面驱动的问答，而是由时间、条件和随机事件共同推动的动态故事。角色的情感不再是固定的人设标签，而是随互动自然变化的内心世界。

第四，从硬编码到可配置。将角色定义、考核方式、事件内容从代码中剥离为配置文件，使得系统具有高度的可扩展性和可复用性。新增角色或考核只需添加配置文件，无需修改任何代码。

第五，从黑箱到透明。所有叙事状态以人类可读的 Markdown 文档存储，用户可以随时查看和手动调整角色的内心世界、角色间的关系、教学进度等。这种透明性赋予了用户对叙事走向的最终控制权，也大大降低了系统的调试和维护成本。